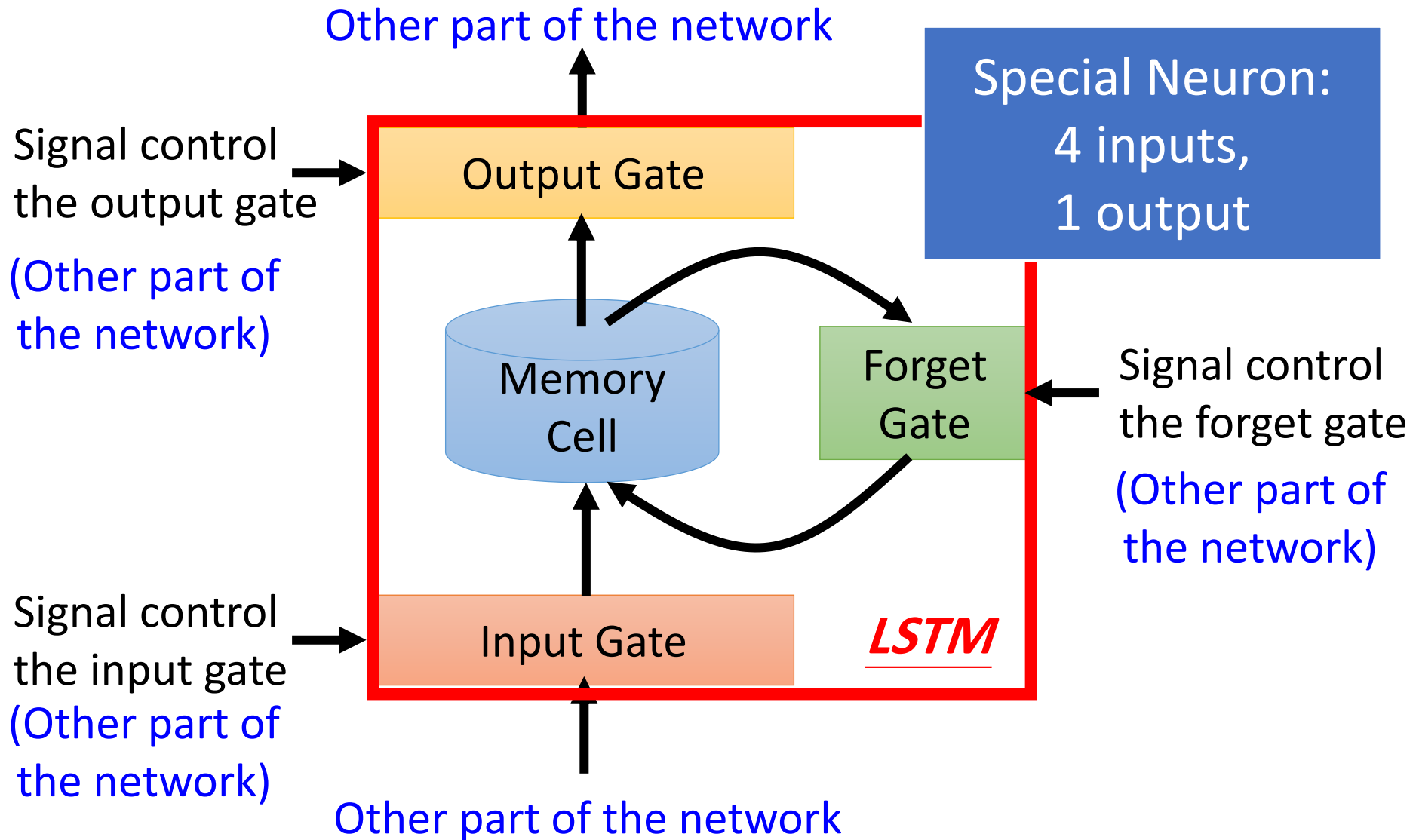
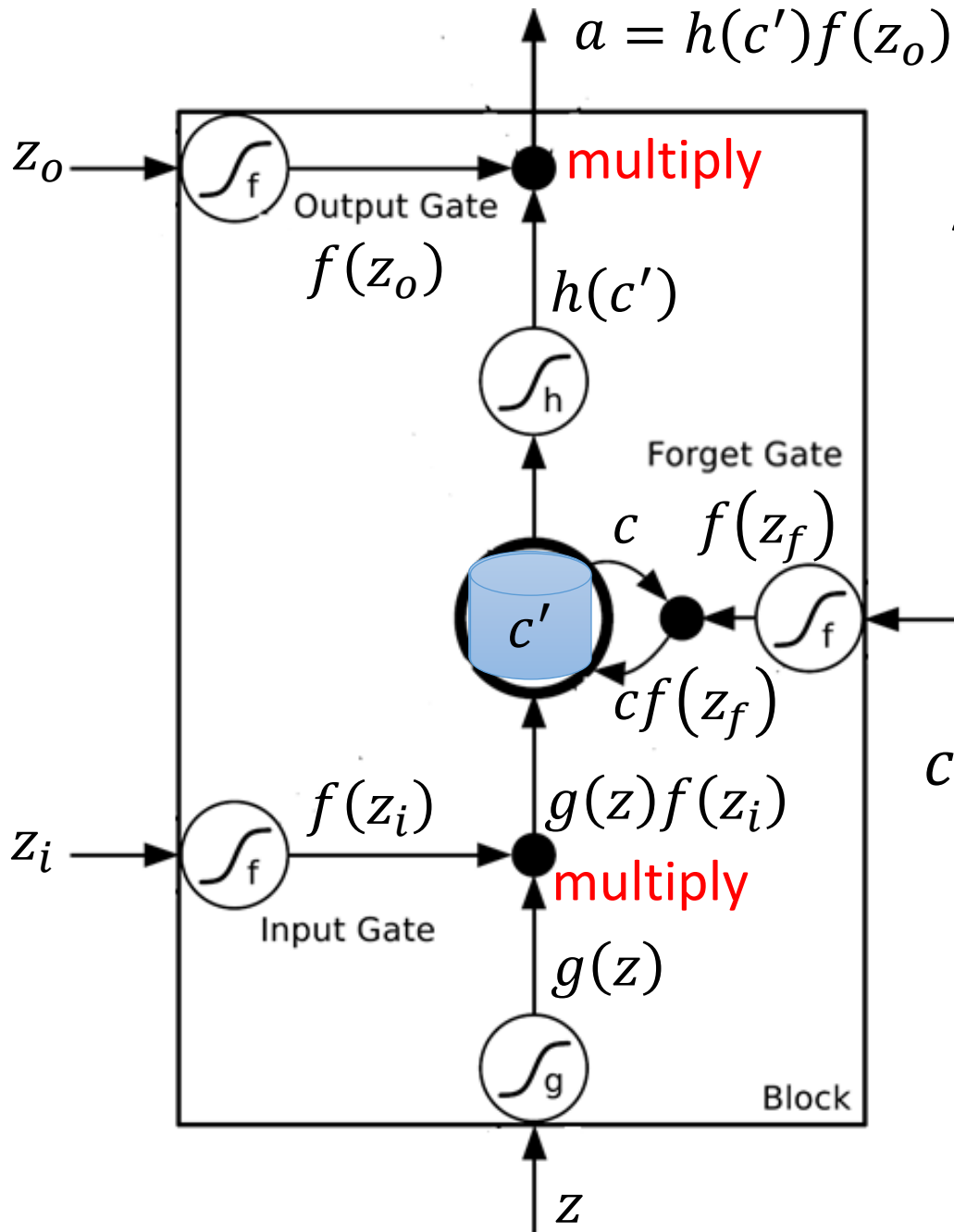


Long Short-term Memory (LSTM)





Activation function f is usually a sigmoid function

Between 0 and 1

Mimic open and close gate

$$c' = g(z)f(z_i) + cf(z_f)$$

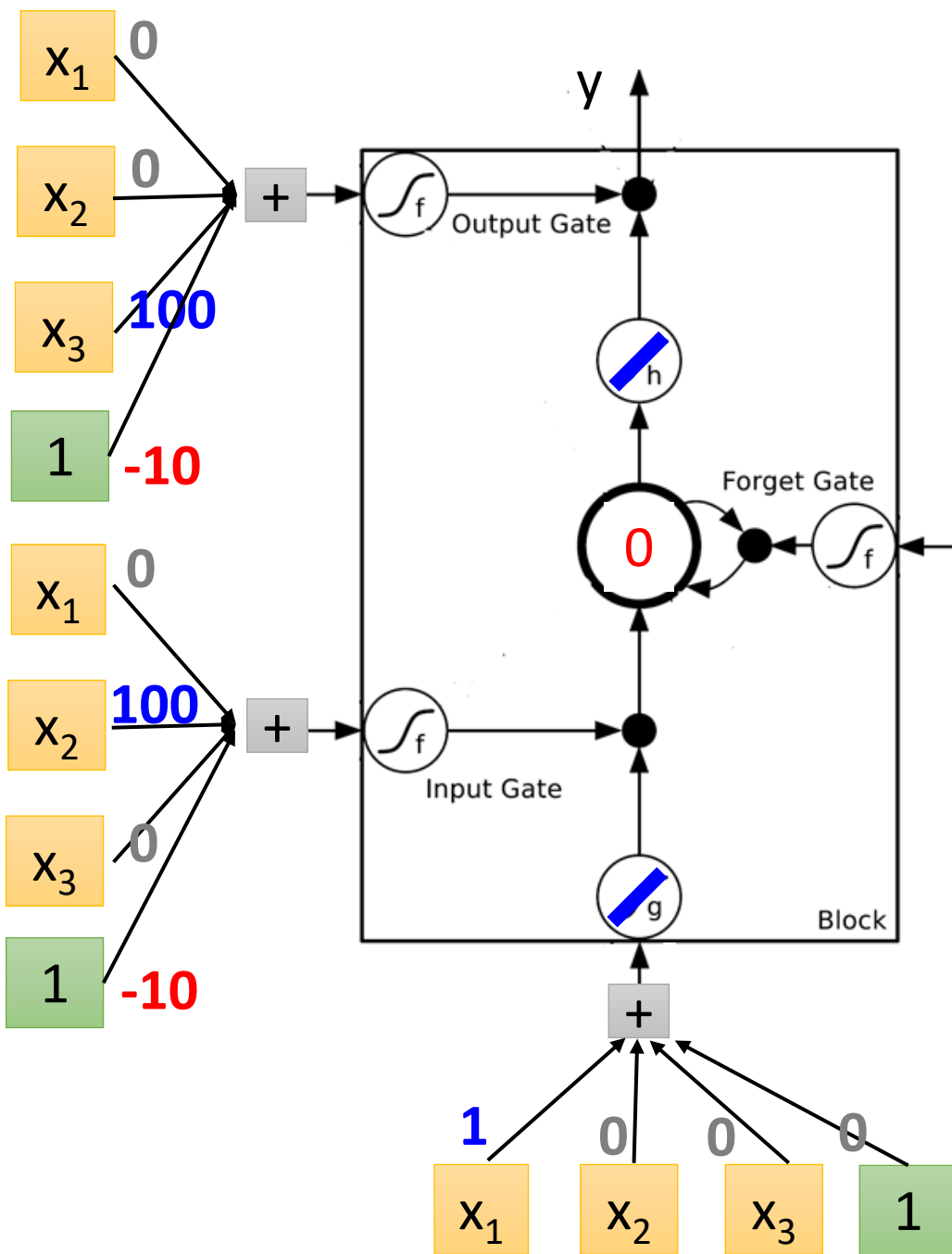
LSTM - Example

	0	0	3	3	7	7	7	0	6
x_1	1	3	2	4	2	1	3	6	1
x_2	0	1	0	1	0	0	-1	1	0
x_3	0	0	0	0	0	1	0	0	1
y	0	0	0	0	0	7	0	0	6

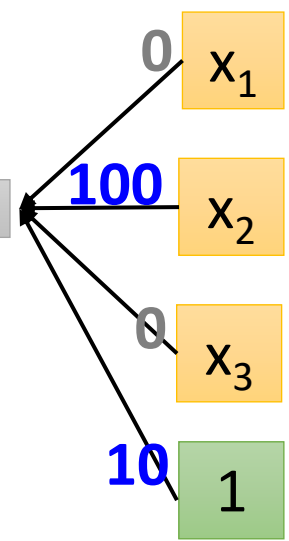
When $x_2 = 1$, add the numbers of x_1 into the memory

When $x_2 = -1$, reset the memory

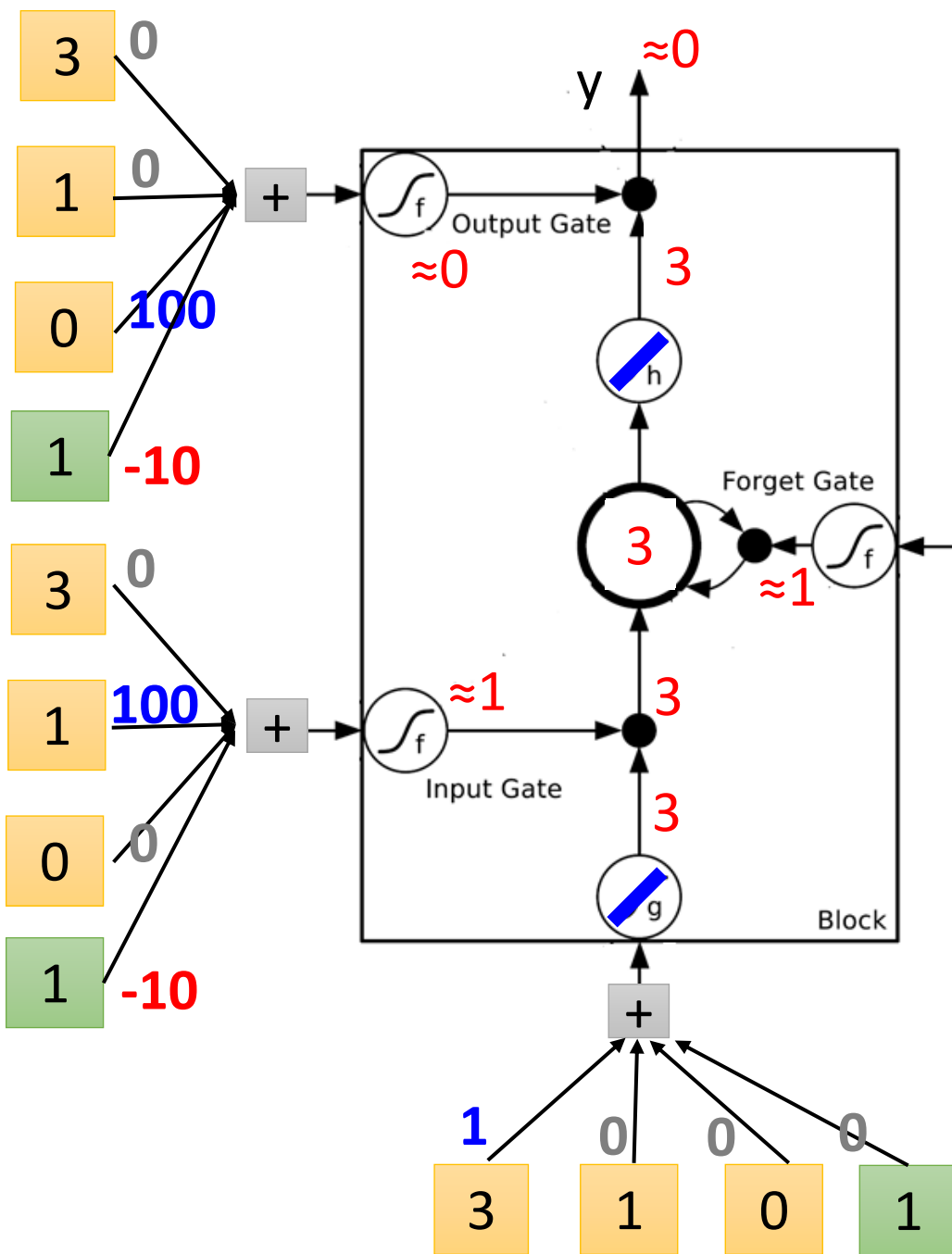
When $x_3 = 1$, output the number in the memory.



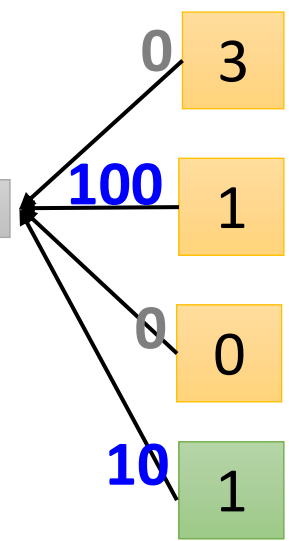
y 0 0 0 7 0



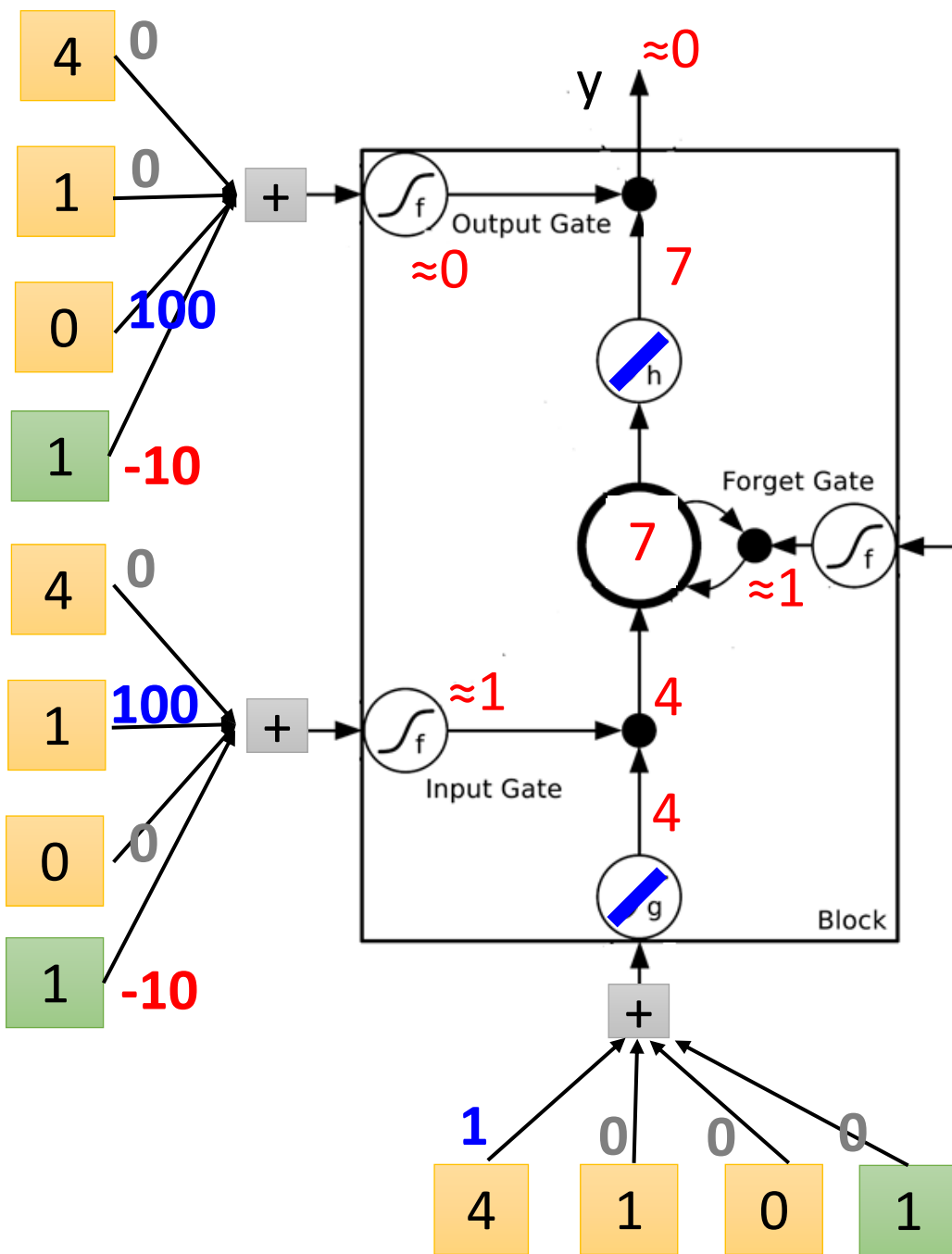
	3	4	2	1	3
x_1	3	4	2	1	3
x_2	1	1	0	0	-1
x_3	0	0	0	1	0



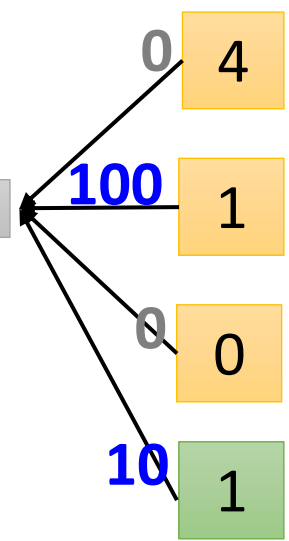
y 0 0 0 7 0



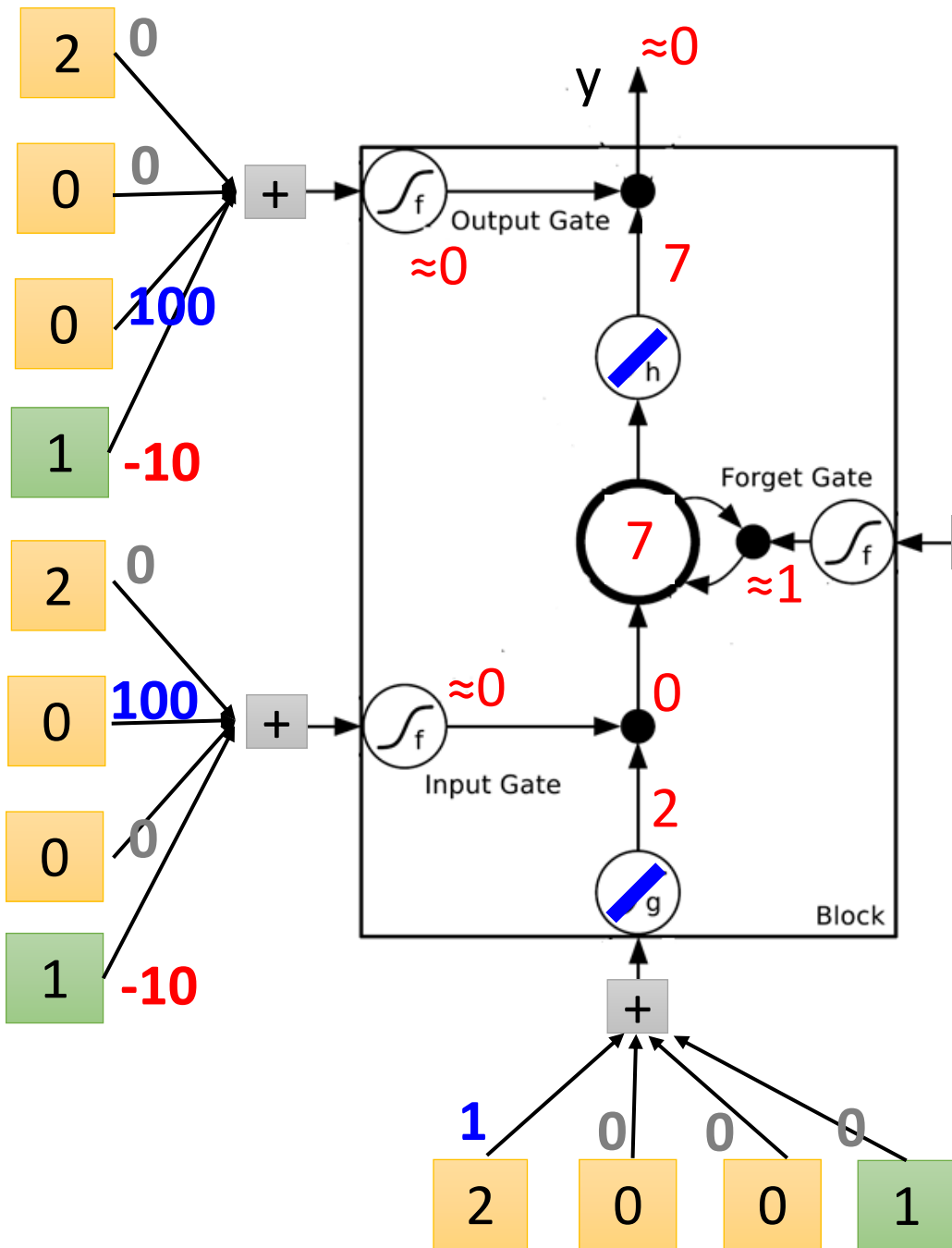
x_1 3 4 2 1 3
 x_2 1 1 0 0 -1
 x_3 0 0 0 1 0



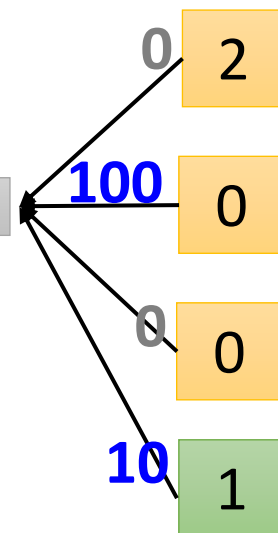
y 0 0 0 7 0



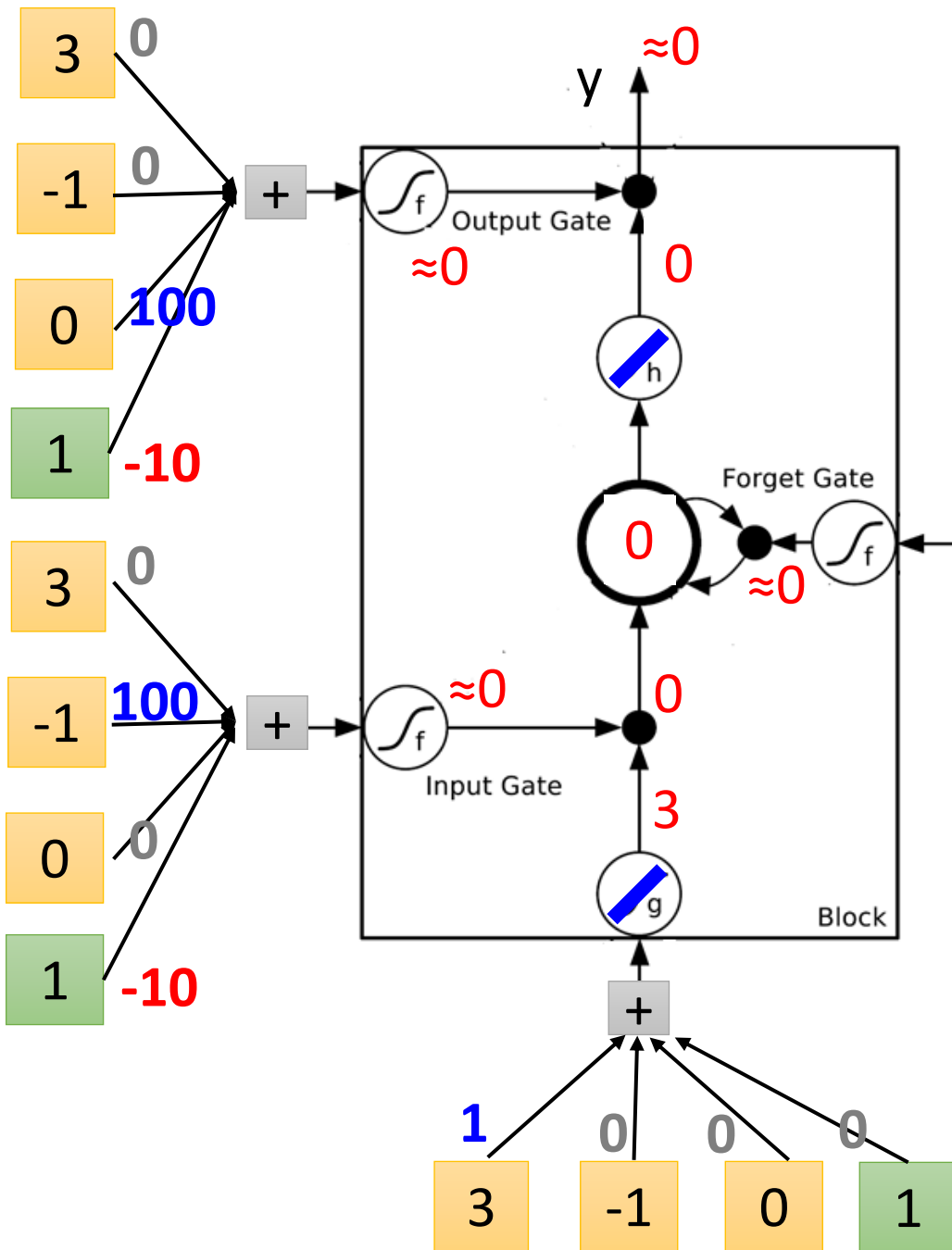
	x_1	x_2	x_3		
	3	4	2	1	3
x_1	1	1	0	0	-1
x_2	0	0	0	1	0



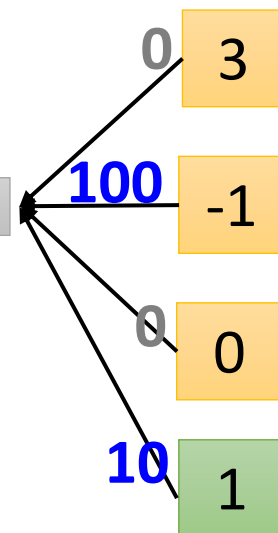
y 0 0 0 7 0



x_1 3 4 2 1 3
 x_2 1 1 0 0 -1
 x_3 0 0 0 1 0



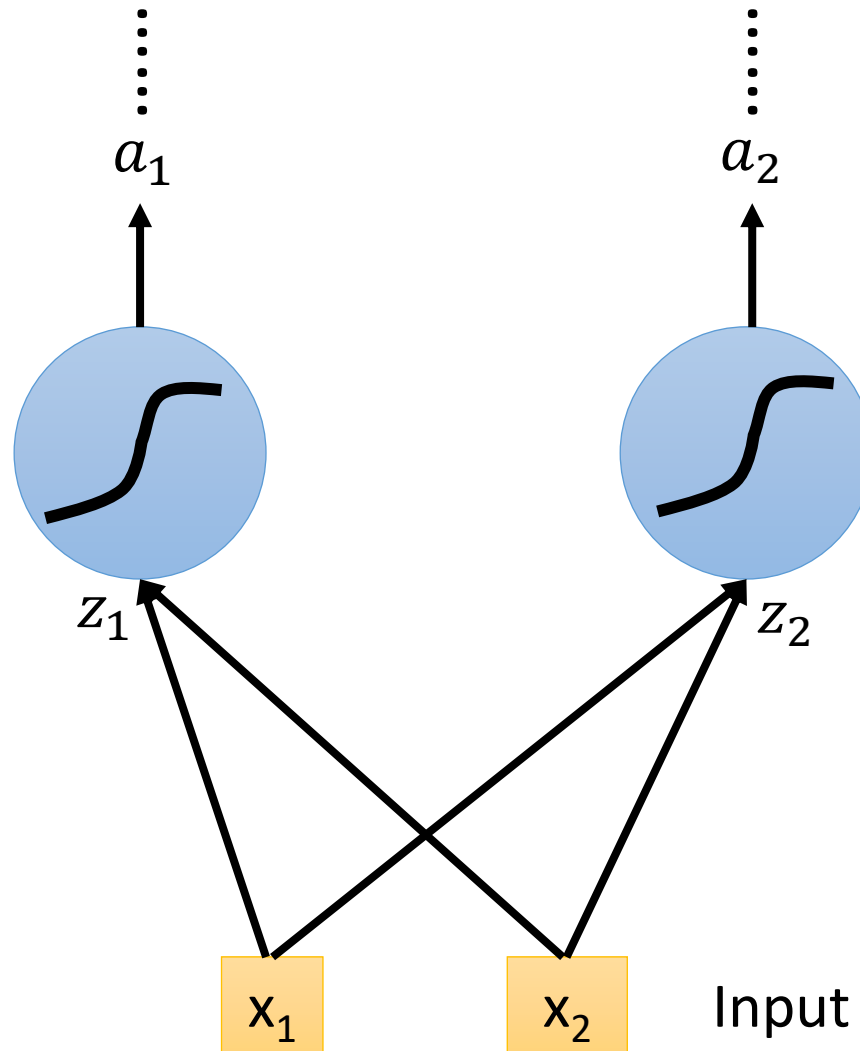
y 0 0 0 7 0

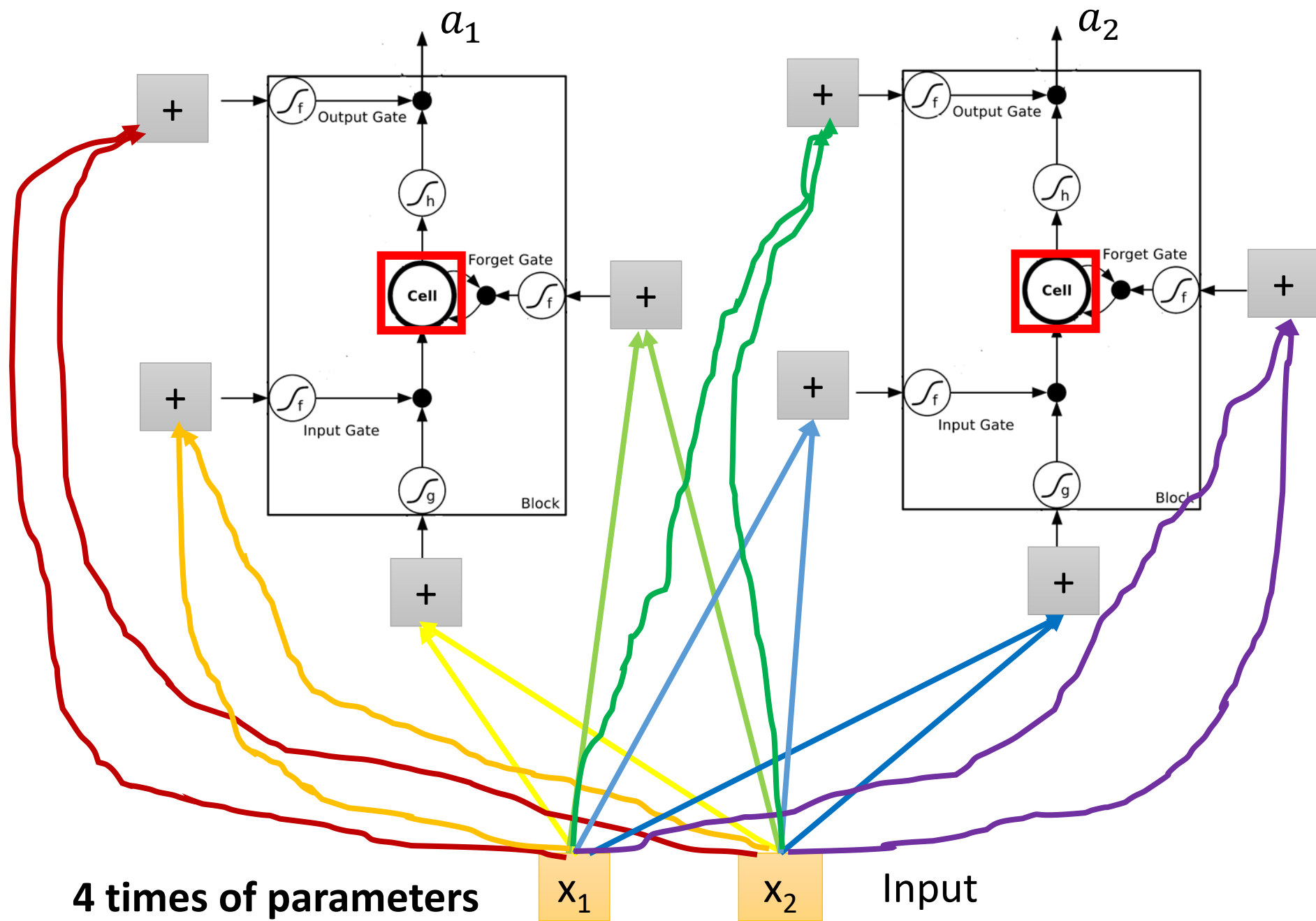


	x_1	x_2	x_3
x_1	3	4	2
x_2	1	1	0
x_3	0	0	0

Original Network:

- Simply replace the neurons with LSTM

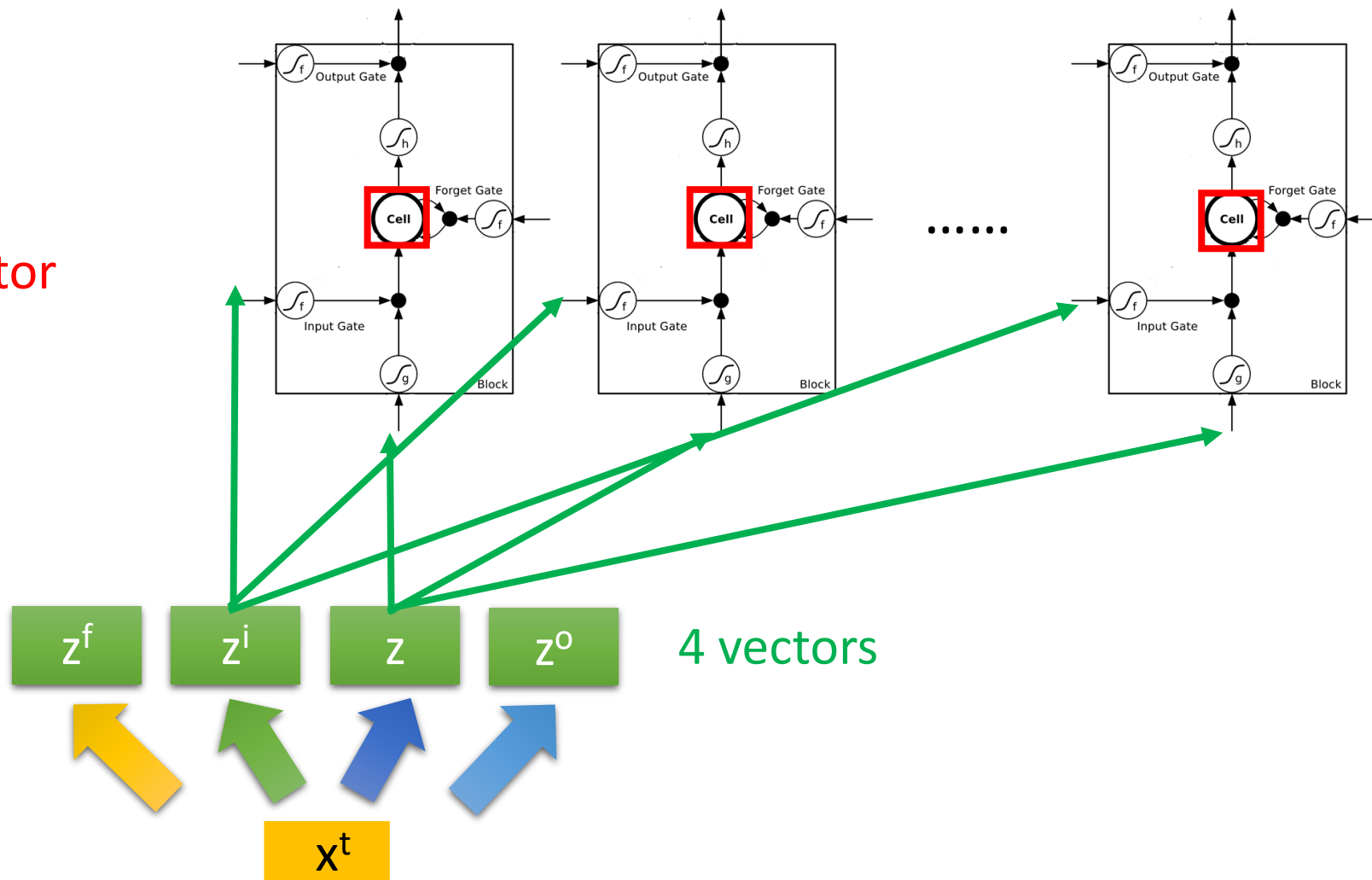




LSTM

c^{t-1}

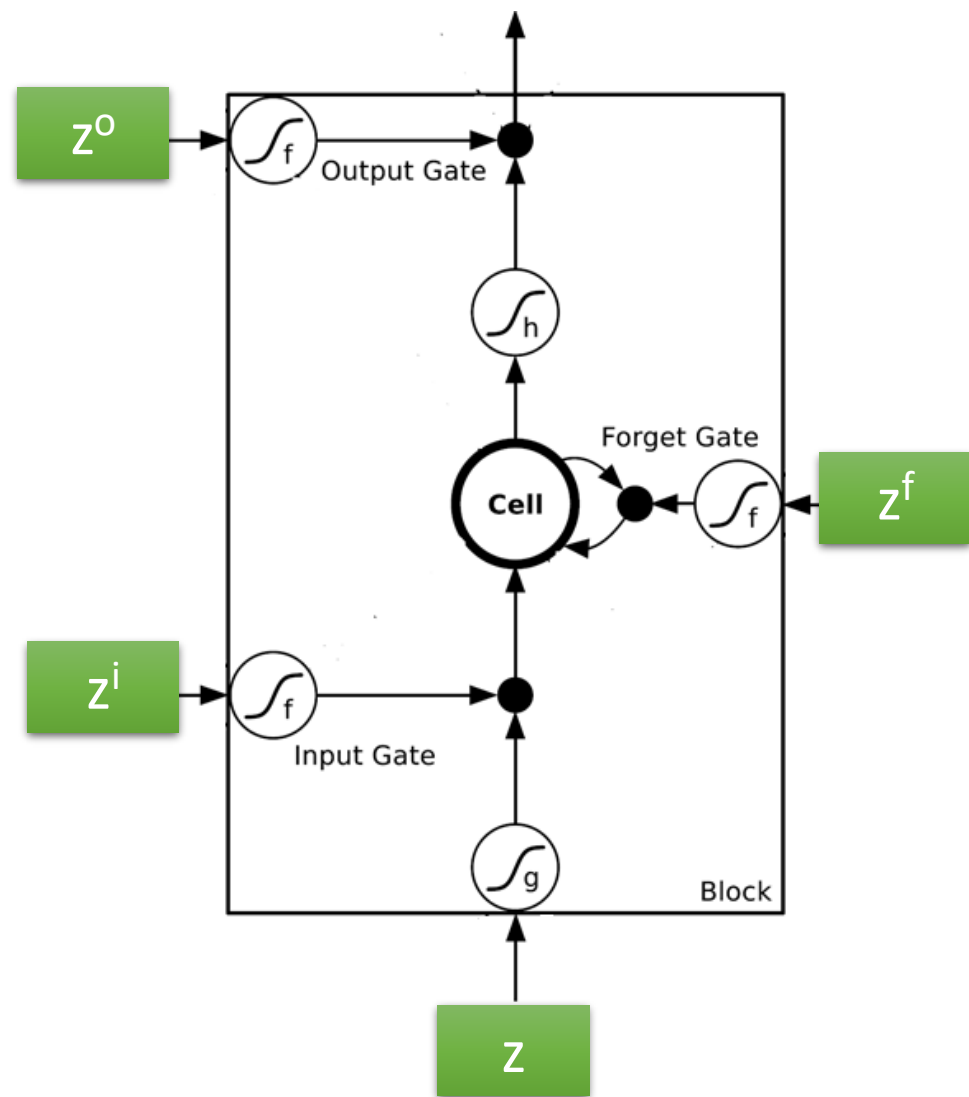
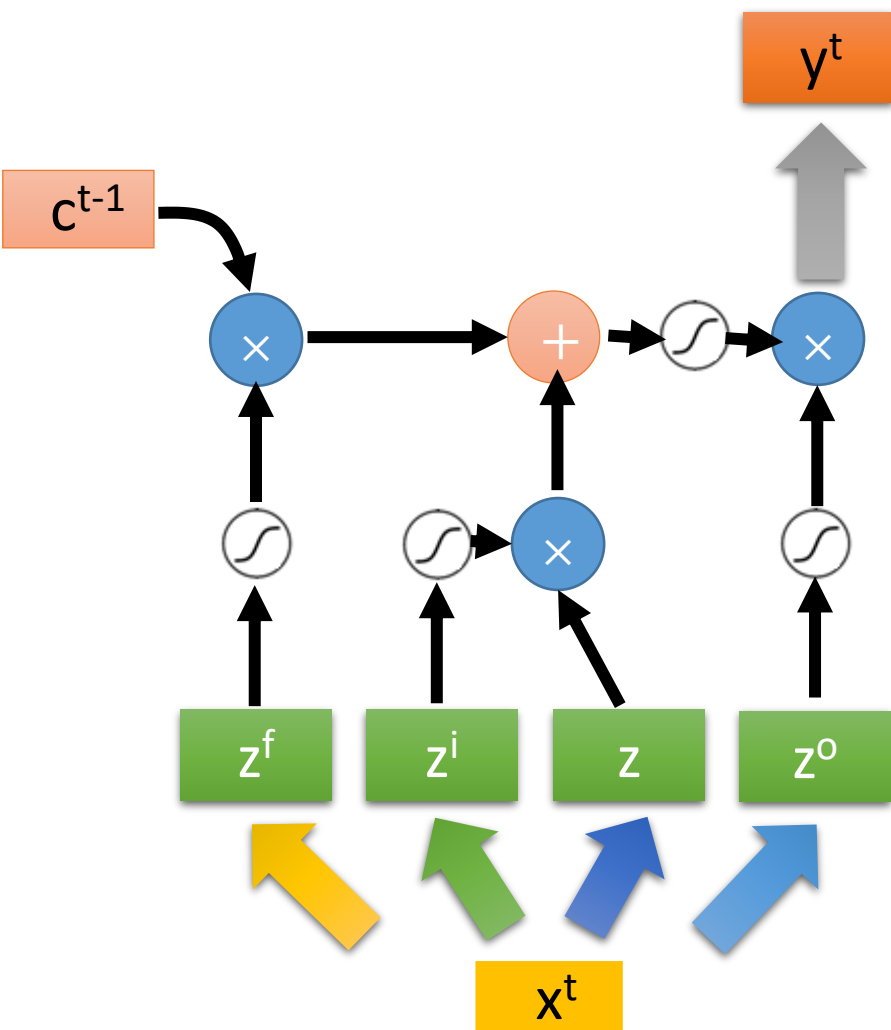
vector



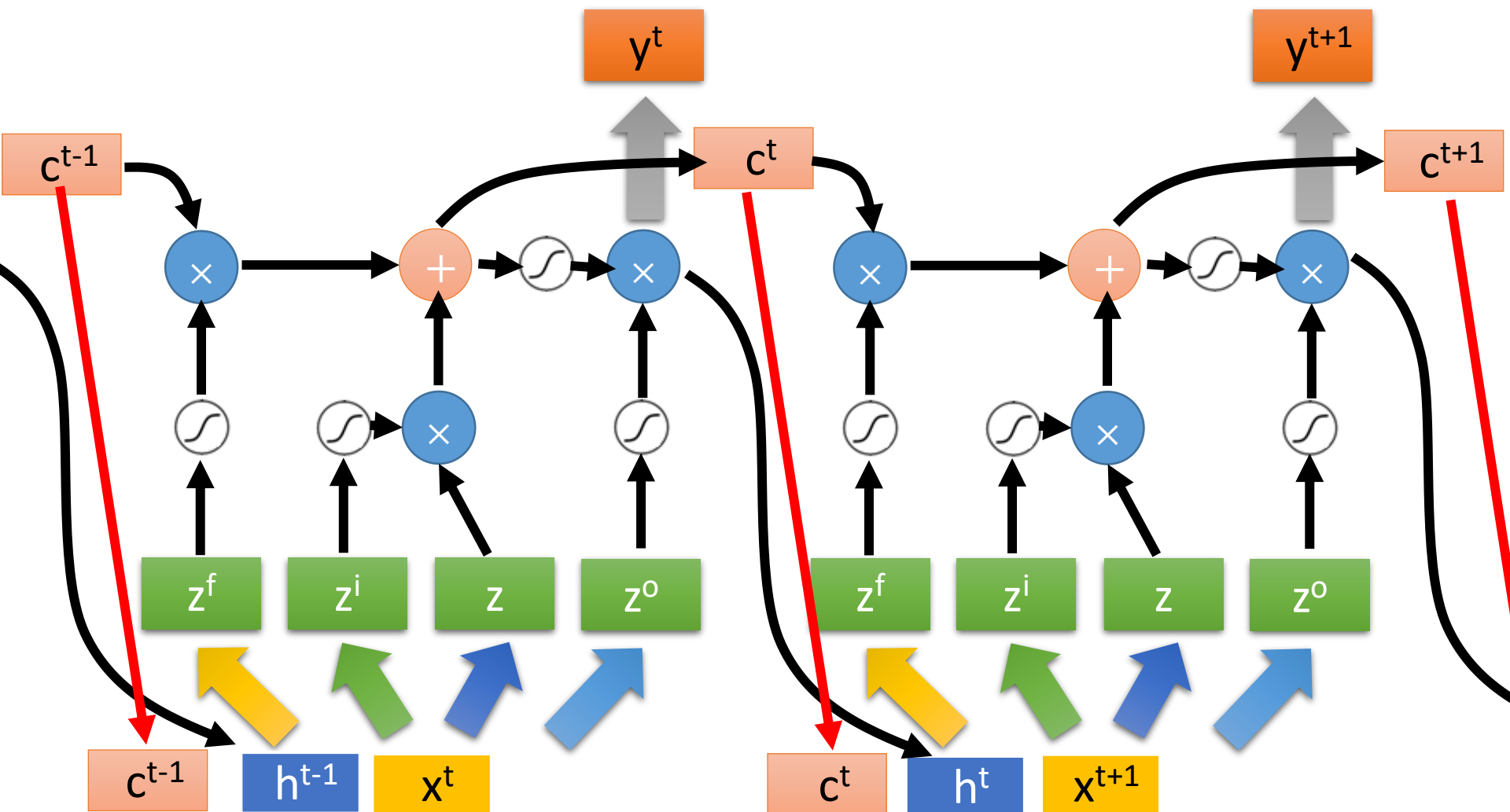
4 vectors

x^t

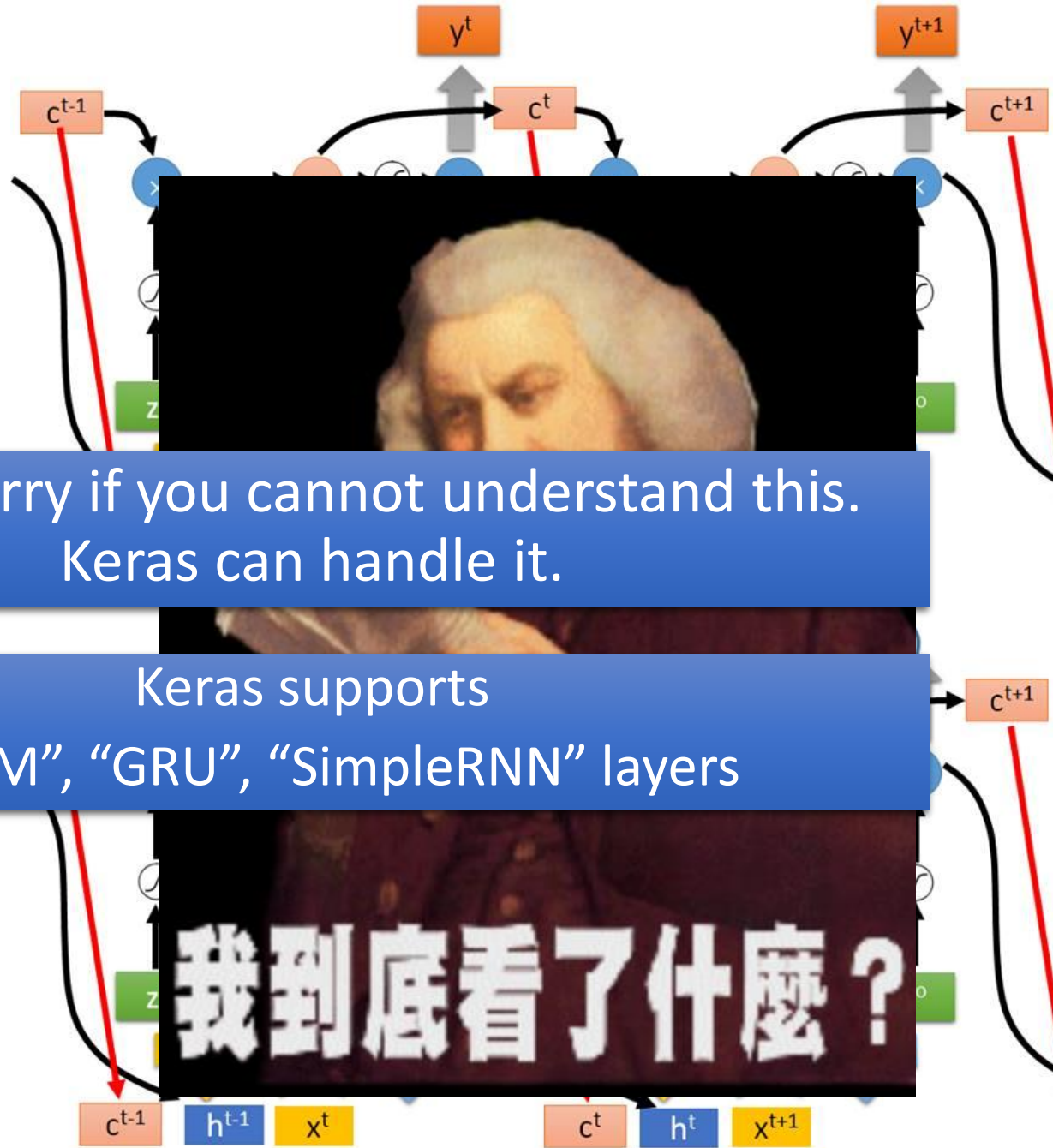
LSTM



LSTM



Multiple-layer LSTM



Don't worry if you cannot understand this.
Keras can handle it.

Keras supports
"LSTM", "GRU", "SimpleRNN" layers

This is quite
standard now.

Helpful Techniques

- Long Short-term Memory (LSTM)

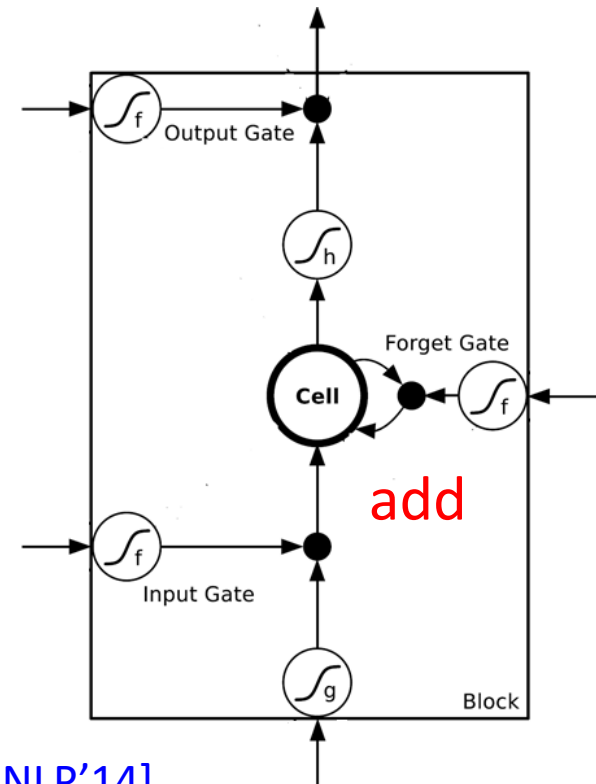
- Can deal with gradient vanishing (not gradient explode)

- Memory and input are *added*

- The influence never disappears unless forget gate is closed

➡ No Gradient vanishing
(If forget gate is opened.)

Gated Recurrent Unit (GRU):
simpler than LSTM



[Cho, EMNLP'14]